JACKAL TRAJECTORY CONTROL

ROS 2 Humble - Installation, commands, parameters, and mission behavior

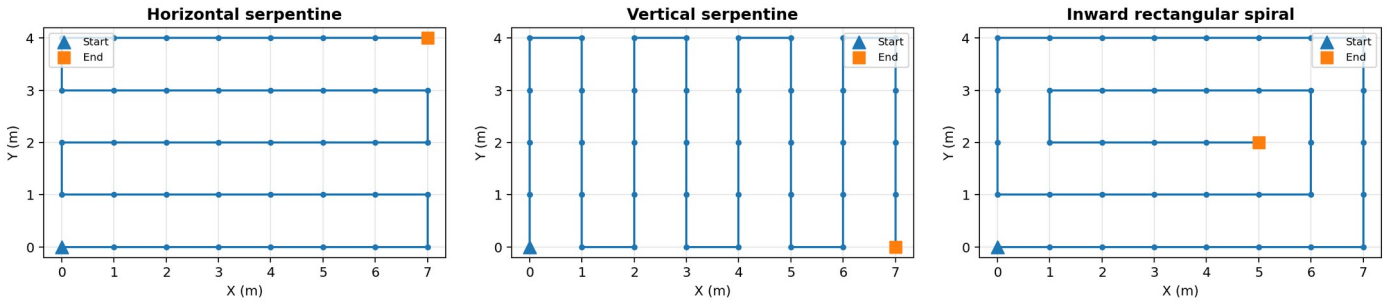
Purpose: Move a Clearpath Jackal through an 8 x 5 horizontal serpentine, vertical serpentine, or inward rectangular spiral using closed-loop odometry feedback.

Mission flow

```
Current pose  ->  Home (x, y, yaw)  ->  Outbound path  ->  Exact reverse path  ->  Home
                        repeat the outbound/return cycle for the requested number of laps
```

- Home defaults to odometry coordinates x=0.0 m, y=0.0 m, yaw=0.0 deg.
- With return_mode=reverse, the robot visits the outbound waypoints in exact reverse order.
- The robot pauses at every outbound and return waypoint.
- At final home arrival, the robot aligns itself with home_yaw_deg.
- The controller publishes Twist commands and does not perform obstacle avoidance.

Available trajectories



Original path preserved: The horizontal trajectory follows the same 40-position 8 x 5 serpentine generated by visualisations.py: even rows move left-to-right and odd rows move right-to-left.

1. Install the ROS 2 package

```
mkdir -p ~/jackal_ws/src
cd ~/jackal_ws/src
# Copy the jackal_trajectory_control folder into this directory.

cd ~/jackal_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

Optional automatic sourcing:

```
echo 'source ~/jackal_ws/install/setup.bash' >> ~/.bashrc
source ~/.bashrc
```

2. Start the empty Gazebo world

```
source /opt/ros/humble/setup.bash
source ~/clearpath/setup.bash

LIBGL_ALWAYS_SOFTWARE=1 ros2 launch clearpath_gz simulation.launch.py world:=empty
```

Gazebo check: Press Play if the world opens paused. Keep this terminal running.

3. Run a trajectory

Horizontal serpentine - one lap and reverse return

```
ros2 launch jackal_trajectory_control trajectory.launch.py \
  trajectory:=horizontal return_mode:=reverse laps:=1 \
  home_x:=0.0 home_y:=0.0 home_yaw_deg:=0.0
```

Vertical serpentine

```
ros2 launch jackal_trajectory_control trajectory.launch.py \
  trajectory:=vertical return_mode:=reverse laps:=1
```

Inward spiral - three laps

```
ros2 launch jackal_trajectory_control trajectory.launch.py \
  trajectory:=spiral return_mode:=reverse laps:=3
```

Recommended first test: smaller and slower

```
ros2 launch jackal_trajectory_control trajectory.launch.py \
  trajectory:=horizontal return_mode:=reverse laps:=1 \
  spacing_x:=0.5 spacing_y:=0.5 \
  max_linear_speed:=0.15 max_angular_speed:=0.40 \
  dwell_seconds:=1.0
```

Mission and home behavior

1. The node waits until it receives nav_msgs/msg/Odometry.
2. If go_home_first=true, it drives directly to (home_x, home_y).
3. At home, it rotates to home_yaw_deg. This yaw also rotates the complete grid.
4. It follows the selected outbound path using odometry feedback.
5. It stops for dwell_seconds at every measurement position.
6. With return_mode=reverse, it follows the exact waypoint sequence backward to home.
7. At home, it restores home_yaw_deg and begins the next lap or stops.

Multiple-lap rule: laps greater than 1 requires return_mode=reverse. This guarantees that each lap begins from the same home pose.

How home works

- home_x and home_y are coordinates in the active odometry frame, not GPS coordinates.
- For the current Gazebo setup, use home_x=0.0 and home_y=0.0.
- On a physical Jackal, place the robot at the desired physical home before starting the odometry/localization session, then keep home at 0,0.
- home_yaw_deg defines both the final heading and the orientation of the trajectory grid.
- Example: home_yaw_deg=90 rotates the entire path 90 degrees counter-clockwise in odom.
- Odometry drifts and may reset after a reboot. A persistent home across sessions requires a future map-frame localization implementation.

Return sequence example

```
Outbound: P0 -> P1 -> P2 -> P3 -> P4
Return:                P3 -> P2 -> P1 -> P0

The current end point P4 is not duplicated. Every return point uses the same dwell time.
```

Stopping safely

- Press Ctrl+C in the trajectory terminal. The node publishes multiple zero-velocity commands before shutdown.
- Do not run teleop_twist_keyboard or another cmd_vel publisher at the same time.
- Keep the emergency-stop method for the real Jackal within immediate reach.

Parameter reference - mission and geometry

Parameter	Default	Purpose
robot_namespace	j100_0000	ROS namespace used by the Jackal. Change it to match the simulated or physical robot.
trajectory	horizontal	horizontal, vertical, or spiral.
return_mode	none	none ends at the last outbound point. reverse returns through the exact path backward.
laps	1	Number of outbound/return cycles. Values above 1 require reverse mode.
home_x	0.0	Home X coordinate in metres in the odometry frame.
home_y	0.0	Home Y coordinate in metres in the odometry frame.
home_yaw_deg	0.0	Home heading and trajectory-grid rotation in degrees.
go_home_first	true	Move to home and align yaw before beginning the first trajectory.
columns	8	Number of X positions in the grid.
rows	5	Number of Y positions in the grid.
spacing_x	1.0	Distance between adjacent X positions in metres.
spacing_y	1.0	Distance between adjacent Y positions in metres.
dwell_seconds	0.5	Stop duration at every outbound and return waypoint.

Useful geometry examples

Use case	Arguments	Result
Full project grid	columns:=8 rows:=5 spacing_x:=1.0 spacing_y:=1.0	7 m x 4 m footprint, 40 positions
Half-size test	columns:=8 rows:=5 spacing_x:=0.5 spacing_y:=0.5	3.5 m x 2 m footprint, 40 positions
Compact test	columns:=4 rows:=3 spacing_x:=0.5 spacing_y:=0.5	1.5 m x 1 m footprint, 12 positions

Parameter reference - controller and ROS topics

Parameter	Default	Purpose
cmd_vel_topic	cmd_vel	Relative Twist topic under robot_namespace. Default resolves to /j100_0000/cmd_vel.
odom_topic	empty	When empty, the node discovers a nav_msgs/msg/Odometry topic automatically.
max_linear_speed	0.30	Maximum forward velocity in m/s.
min_linear_speed	0.06	Minimum nonzero forward command in m/s.
max_angular_speed	0.70	Maximum yaw velocity in rad/s.
linear_gain	0.80	Proportional gain applied to waypoint distance.
angular_gain	1.80	Proportional gain applied to heading and yaw errors.
position_tolerance	0.08	Waypoint acceptance radius in metres.
yaw_tolerance_deg	3.0	Accepted final home-yaw error in degrees.
turn_in_place_angle_deg	31.5	Heading error above which the robot rotates before advancing.
control_rate	20.0	Controller update rate in hertz.
odom_timeout	1.0	Stop if odometry is older than this number of seconds.

Diagnostics

```
ros2 topic list -t | grep Odometry
ros2 topic list | grep cmd_vel
ros2 node list | grep trajectory_follower
ros2 topic echo /j100_0000/cmd_vel --once
```

Common problems

Symptom	Action
No odometry topic found	Run ros2 topic list -t | grep Odometry, then pass odom_topic explicitly or confirm that simulation/controllers are active.
Robot does not move	Confirm Gazebo is playing, the namespace is correct, and no emergency-stop or higher-priority twist_mux input is active.
Robot starts from the wrong place	Verify that home_x/home_y match the active odom frame. For the current simulation use 0,0.
Path is rotated incorrectly	Set home_yaw_deg to the desired grid orientation. Use 0 deg for +X, 90 deg for +Y.
Node rejects laps > 1	Use return_mode:=reverse. Multiple laps are intentionally blocked without a home return.
Corners are too wide	Reduce max_linear_speed or increase turn-in-place behavior by lowering turn_in_place_angle_deg.

Real-robot safety: This is a waypoint controller, not a navigation stack. Test at low speed in a clear, bounded area before attaching the CR350 or running a full 7 m x 4 m mission.